

Laboratorio di Ingegneria Informatica 2026

Lorenzo Castelli
2115498

Ludovica Fiocchetta
2103669

Alessandro Genovese
2108670

1 Introduzione

Il progetto illustra lo sviluppo di una piattaforma architetturale a microservizi finalizzata all'estrazione automatizzata (parsing), all'archiviazione e alla valutazione qualitativa e quantitativa di documenti web.

L'obiettivo principale è l'estrazione di testo pulito delle pagine di quattro domini internet tramite la libreria **Crawl4AI**, e la successiva creazione di un ecosistema scalabile capace di misurare l'accuratezza dei parser sviluppati rispetto a dataset di riferimento, i **gold standard**, sia mediante metodologie di **Intelligenza Artificiale (LLM-as-a-Judge)**, sia di metriche sintattiche deterministiche.

I domini di riferimento sono:

- en.wikipedia.org
- www.viaggi-usa.it
- www.cbsnews.com
- www.cnbc.com

2 Organizzazione del codice

Il progetto è strutturato separando la logica di estrazione, valutazione e salvataggio dei dati (**Backend**) dall'interfaccia utente (**Frontend**). L'intero sistema è containerizzato mediante Docker, tramite le specifiche in `docker-compose.yaml` per garantire la portabilità e riproducibilità dell'ambiente di esecuzione.

2.1 Backend

Il backend, esposto sulla porta 8003, è stato sviluppato seguendo un'architettura modulare che si articola in tre macro-componenti:

- **Validazione Dati e Server (server.py):** Questo modulo espone i servizi verso il **Frontend** sfruttando il framework **FastAPI**. Implementa l'interfaccia **REST API** e utilizza modelli **Pydantic** per la validazione rigorosa dei dati in ingresso e in uscita, assicu-

rando che ogni endpoint (`/parse`, `/domains`, `/gold_standard`, ecc.) rispetti le specifiche richieste. La logica di routing smista dinamicamente le richieste ai diversi parser in base al dominio dell'URL fornito.

- **Pipeline di Parsing (crawler_*.py):** Ogni dominio è gestito da un modulo dedicato che implementa una pipeline di estrazione a tre stadi basata sulla libreria **Crawl4AI**:

1. *Configurazione e Acquisizione:* Viene utilizzato **BrowserConfig** in modalità **headless** e **CrawlerRunConfig** con **CacheMode.BYPASS** per garantire l'acquisizione in tempo reale del contenuto. Se viene specificata in input una stringa **html**, viene invece svolta l'estrazione staticamente da quest'ultima senza interrogazione web.
2. *DOM Cleaning tramite JavaScript:* Prima della conversione in **Markdown**, viene iniettato codice **JavaScript** (`js_code`) per manipolare il **DOM** e rimuovere selettivamente elementi di disturbo come **infobox**, **sidebar**, **footer** e **banner pubblicitari**.
3. *Post-processing Regex:* Il testo viene convertito in **Markdown** e ulteriormente filtrato tramite **espressioni regolari** per eliminare residui di tag, metadati, spazi bianchi ridondanti, restituendo un testo pulito pronto per l'analisi.

- **accuracy_.py:** Questo modulo funge da motore di valutazione del sistema. È stato progettato come un servizio indipendente che riceve stringhe di testo (estratto e di riferimento) e calcola il grado di sovrapposizione tra i set di token, operando una normalizzazione preventiva (lowercase e rimozione della punteggiatura) per garantire l'oggettività del confronto.
- **init_db.py:** Modulo di inializzazione della

Base di Dati.

- **remove_markdown.py**: helper per la rimozione di markdown.

2.2 Frontend

L'interfaccia utente è stata progettata per fornire un punto di accesso intuitivo alle funzionalità del sistema. Il modulo è esposto sulla porta 8004 e si compone delle seguenti parti:

- **frontend.py**: Sviluppato con **FastAPI**, agisce come un'interfaccia tra l'utente e il backend. Utilizza il client asincrono **httpx** per comunicare internamente con i servizi del backend, recuperando dinamicamente la lista dei domini supportati, gli URL presenti nel Gold Standard e lo stato dei componenti (Backend, MariaDB e Ollama) in tempo reale, gestendo così l'intero ciclo di richiesta-risposta.
- **templates/**: La presentazione dei dati è affidata a templates HTML che utilizzano il motore di rendering **Jinja2** per la generazione dinamica dei contenuti. Grazie a Jinja2, l'interfaccia è in grado di adattarsi ai dati ricevuti: mostra messaggi di errore contestuali e si articola in diverse aree specializzate: una home page di riepilogo dello stato, un'interfaccia di parser con tre colonne per facilitare il confronto testuale tra Row HTML, Gold Standard e testo parsato dell'url preso in input, con sotto le relative metriche, un pannello di amministrazione per l'inserimento o la rimozione di Gold Standard, ed infine una pagina analitica per vedere le statistiche complessive di ciascun dominio, sia metriche base e avanzate che giudizio dell'LLM.
- **static/**: Questa directory contiene il foglio di stile **CSS** che definisce l'estetica dell'applicazione.

2.3 Dati e Configurazione

- **domains.json**: File di configurazione che elenca i domini di riferimento.
- **gs_data/**: Contiene, in formato JSON, i Gold Standard necessari per il calcolo delle metriche di accuratezza.

3 Database

Per garantire la persistenza dei dati ottenuti, l'architettura fa uso di un database integrato alla pipeline mediante MariaDB. Questa scelta consente una maggiore velocità nello svolgimento di

valutazioni successive di una determinata pagina web, poichè il sistema è in grado di riutilizzare l'HTML precedentemente salvato, evitando di doverlo ricaricare ripetutamente dalla rete.

3.1 Tabelle relazionali

Le tabelle relazionali scelte sono: (N.B. l'attributo sottolineato rappresenta la chiave primaria)

- **web_resources (url, domain, title, html_text, created_at)**: Permette di memorizzare dai Gold Standard l'HTML delle pagine web richieste dalla specifica del progetto, insieme al titolo e al dominio.
- **gold_standard (url, gold_text, created_at)**: Conserva il testo di riferimento prelevato dai Gold Standard.
- **parsed_pages table (url, parsed_text, created_at)**: Permette la memorizzazione del testo estratto dal crawler.
- **evaluation_results table (url, token_level_metrics, LLM_judgments, evaluated_at)**: Archivia le metriche calcolate per l'URL target. In particolare LLM_JUDGMENTS fa riferimento alle metriche descritte nella sezione 4, TOKEN_LEVEL_METRICS a quelle nella sezione 5.

3.2 Gestione Tabelle

Il database viene inizializzato al primo avvio del progetto, popolando di default le tabelle `web_resources`, `gold_standard` e `evaluation_results` utilizzando i dati presenti in `/gold_standard`. La tabella `evaluation_results` viene popolata con le metriche a NULL, valori che verranno aggiornati al seguito di richieste di valutazioni. Vengono inoltre modificate in seguito a operazioni di parsing sia la tabella `web_resources` che `parsed_pages`. Infine, sono stati anche implementati endpoint specifici per l'eliminazione di record all'interno di `web_resources` e `gold_standard`.

4 Valutazione dei Judge

Per l'implementazione del paradigma *LLM-as-a-Judge* è stato selezionato il modello `llama3.2`, eseguito localmente tramite l'infrastruttura Ollama per garantire efficienza computazionale e indipendenza da API esterne. L'obiettivo è la valutazione dell'accuratezza semantica del testo estratto dai parser (*Parsed Text*) tramite il confronto con il testo di riferimento (*Gold Standard*).

Il bilanciamento e l'ottimizzazione del prompt sono stati strutturati seguendo criteri metodologici rigorosi per rispondere a due problematiche principali:

- **Gestione del contesto e dei tempi di calcolo:** Data la potenziale instabilità o latenza causata da stringhe eccessivamente lunghe, è stato applicato un limite stringente di 1000 caratteri sia per il testo estratto sia per il gold standard, preservando così la stabilità dell'inferenza senza compromettere la valutazione semantica della porzione principale.
- **Ingegneria del Prompt e Formattazione:** Il prompt è stato progettato per forzare il modello a comportarsi come un valutatore esperto di *Information Extraction*. Viene richiesta l'assegnazione di un punteggio intero da 1 (completamente errato/mancante) a 5 (perfettamente coerente e accurato).

Per ovviare alla natura tipicamente stocastica dei Large Language Models e per consentire una pipeline di estrazione dati robusta, la configurazione dei parametri di generazione ha previsto l'impostazione della temperatura a 0.0, garantendo la massima determinazione e replicabilità dei giudizi. Inoltre, è stata imposta la modalità `format="json"` per obbligare il modello a restituire unicamente un oggetto strutturato contenente le chiavi `judge_score` e `judge_feedback`.

5 Valutazione parser

La qualità dei parser è stata misurata confrontando l'output testuale ottenuto da ogni dominio con i dati di riferimento contenuti nella cartella `gs_data/` (Gold Standard).

5.1 Metodologia di valutazione

Nel modulo `accuracy.py`, oltre alla valutazione obbligatoria *token-level* (Precision, Recall e F1-score) basata sull'intersezione degli insiemi di parole, sono state implementate tre metriche avanzate (`x_eval`) per ottenere un'analisi più robusta:

- **Jaccard Similarity:** Valuta la similarità tra due insiemi testuali calcolando il rapporto tra la dimensione della loro intersezione e la dimensione della loro unione ($|A \cap B| / |A \cup B|$). Rispetto all'F1-score, penalizza in modo più severo l'aggiunta di rumore (falsi positivi) combinata alla perdita di testo (falsi negativi),

fornendo un indicatore molto rigoroso della sovrapposizione del vocabolario.

- **Bigram Overlap:** Calcola la sovrapposizione di coppie di parole consecutive (bigrammi). È fondamentale per verificare se la struttura sintattica locale e l'ordine logico delle frasi sono stati preservati durante il parsing, penalizzando rimescolamenti o frammentazioni eccessive del testo estratto.
- **Cosine Similarity:** Misura l'affinità semantica trasformando i documenti in vettori matematici. Attraverso il modello TF-IDF (*Term Frequency-Inverse Document Frequency*), ogni termine riceve un peso basato sulla sua importanza relativa nel testo. La metrica calcola geometricamente il coseno dell'angolo formato dal vettore del testo parsato e quello del Gold Standard. Questa interpretazione vettoriale rende la metrica robusta per valutare se i documenti sono focalizzati sugli stessi concetti chiave, ignorando le piccole variazioni di punteggiatura o stop-words.

5.2 Risultati ottenuti

Nelle Tabelle sono riportati i risultati medi ottenuti per ciascun dominio target sull'intero Gold Standard, generati tramite l'endpoint `/full_gs_eval`. I valori permettono di quantificare oggettivamente l'efficacia dei parser.

Dominio	Precision	Recall	F1-score
en.wikipedia.org	0.9979	0.9989	0.9984
www.cbsnews.com	1.0000	1.0000	1.0000
www.cnn.com	0.9995	0.9985	0.9990
www.viaggi-usa.it	0.9919	0.9989	0.9953

Dominio	Jaccard S.	Bigram O.	Cosine S.
en.wikipedia.org	0.9969	0.9983	0.9998
www.cbsnews.com	1.0000	1.0000	0.9999
www.cnn.com	0.9981	0.9992	0.9986
www.viaggi-usa.it	0.9909	0.9976	0.9988

Table 1: Metriche di accuratezza Token-level (sopra) e metriche avanzate (sotto) ottenute dai parser per dominio.

Dominio	LLM Judge Score
en.wikipedia.org	4.4
www.cbsnews.com	4.5
www.cnn.com	4.6
www.viaggi-usa.it	5.0

Table 2: Valutazione semantica media assegnata dall'LLM Judge (llama3.2) per ciascun dominio analizzato.